

Université / Établissement

École Supérieure de Technologie de Tétouan

Rapport de TP

Exploration de l'espace des états : Recherche Aveugle

Étudiant : SEBAGH Badr Eddine

Encadrant / Professeur : Faouzi Marzouki

Module : Introduction IA

Année universitaire : 2025/2026

Département d'Informatique

Table des matières

1	Introduction	2
2	Algorithmes Implémentés	2
2.1	Breadth-First Search (BFS)	2
2.2	Depth-First Search (DFS)	3
3	Implémentation et Environnement Technique	4
3.1	Représentation du Labyrinthe	5
3.2	Fonctions Clés	5
3.3	Phase d'Exécution	7
4	Tests et Résultats	9
4.1	Scénarios de Test (10×10 vs 15×15)	9
4.2	Visualisation des Performances	10
4.2.1	Résultats visuels (Graphiques)	10
4.3	Tableau Récapitulatif Global	11
5	Analyse Comparative (Étude de la grille 10×10)	11
5.1	Analyse du Temps d'Exécution	12
5.2	Analyse de la Consommation Mémoire	12
5.3	Qualité du Chemin (Optimalité)	12
6	Conclusion Générale	13

1 Introduction

Le problème de la résolution de labyrinthes représente un défi classique et fondamental dans le domaine de l'**intelligence artificielle** et de la **théorie des graphes**. L'objectif principal de ce projet est de concevoir un système capable de trouver un chemin entre un point de départ et une destination, tout en naviguant à travers des obstacles potentiellement complexes.

Objectifs de l'étude :

- réaliser une étude comparative entre deux algorithmes de recherche fondamentaux ;
- évaluer la capacité de chaque méthode à résoudre efficacement un labyrinthe ;
- analyser leurs performances selon des critères quantitatifs précis.

Cette étude repose sur une approche comparative entre les algorithmes suivants :

- **Breadth-First Search (BFS)** : exploration par couches permettant de garantir l'optimalité du chemin lorsque les coûts de déplacement sont uniformes ;
- **Depth-First Search (DFS)** : exploration en profondeur privilégiant, dans certains cas, une découverte rapide d'une solution sans garantir qu'elle soit minimale.

La comparaison se focalise principalement sur deux métriques de performance critiques :

- **la vitesse d'exécution** : le temps nécessaire pour atteindre la sortie ;
- **l'efficacité mémoire** : l'espace de stockage consommé par les structures de données durant l'exploration.

Outils techniques :

- **Python** : comme langage de programmation principal ;
- **time** et **tracemalloc** : pour la mesure précise des performances temporelles et spatiales ;
- **Matplotlib** : pour la génération de graphiques comparatifs facilitant l'analyse visuelle des résultats.

2 Algorithmes Implémentés

Dans cette section, nous détaillons le fonctionnement logique des deux méthodes de recherche implémentées. L'objectif est de comprendre comment chaque algorithme explore la structure de données du labyrinthe.

2.1 Breadth-First Search (BFS)

L'algorithme **BFS** explore le labyrinthe **niveau par niveau**, c'est-à-dire par couches successives.

Logique de fonctionnement :

- il utilise une structure de **file** (FIFO – First In First Out) pour stocker les nœuds à visiter ;

- il explore tous les voisins directs d'un point avant de passer aux voisins du niveau suivant ;
- la progression se fait donc de manière uniforme à travers les différentes couches du labyrinthe.

Avantage :

- cet algorithme garantit de trouver le **chemin le plus court** dans un graphe non pondéré comme notre labyrinthe.

Inconvénient :

- il est gourmand en mémoire, car il doit stocker simultanément tous les nœuds appartenant à la frontière d'exploration.

```
# BFS
def bfs(start, goal):
    queue = deque([start])
    came_from = {start: None}

    while queue:
        current = queue.popleft()
        if current == goal:
            break
        for neighbor in get_neighbors(current):
            if neighbor not in came_from:
                came_from[neighbor] = current
                queue.append(neighbor)

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = came_from[node]
    return path[::-1]
```

FIGURE 1 – Code de l'algorithme BFS

2.2 Depth-First Search (DFS)

À l'inverse, l'algorithme **DFS** explore une branche du labyrinthe le plus loin possible avant de faire marche arrière (*backtracking*).

Logique de fonctionnement :

- il utilise une structure de **pile** (LIFO – Last In First Out) ;

- l'algorithme plonge dans la profondeur du labyrinthe jusqu'à atteindre un cul-de-sac ou l'objectif;
- lorsqu'aucune progression n'est possible, il revient en arrière pour explorer une autre branche.

Avantage :

- il est généralement plus économe en mémoire, car il ne stocke que le chemin actuel et les options non encore explorées associées à ce chemin.

Inconvénient :

- il ne garantit pas de trouver le chemin le plus court et peut s'arrêter dès qu'une solution, même sous-optimale, est trouvée.

```
# DFS
def dfs(start, goal):
    stack = [start]
    came_from = {start: None}

    while stack:
        current = stack.pop()
        if current == goal:
            break
        for neighbor in get_neighbors(current):
            if neighbor not in came_from:
                came_from[neighbor] = current
                stack.append(neighbor)

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = came_from[node]
    return path[::-1]
```

FIGURE 2 – Code de l'algorithme DFS

3 Implémentation et Environnement Technique

Dans cette partie, nous décrivons la mise en œuvre logicielle de notre solution ainsi que l'environnement de test utilisé pour conduire l'étude.

3.1 Représentation du Labyrinthe

Pour modéliser notre environnement, nous avons utilisé un dictionnaire Python représentant une grille de 10×10 .

Choix de la structure (Dictionnaire vs Liste) :

- nous avons privilégié le dictionnaire car il permet un accès en temps constant $O(1)$;
- contrairement à une liste de listes, le dictionnaire permet de vérifier instantanément si une coordonnée (x, y) existe et quel est son état;
- ce choix contribue à optimiser la performance globale de la recherche.

Logique des états :

- la valeur **0** représente un passage libre (*Passable*);
- la valeur **1** représente un mur (*Obstacle*).

Configuration des obstacles :

- nous avons injecté manuellement une liste de coordonnées (**walls**) afin de créer des obstacles complexes;
- cela permet de tester la robustesse des algorithmes face à des chemins étroits ou contraints.

```
# CREATING A 10*10 MAZE :
MY_MAZE = {(r, c): 0 for r in range(10) for c in range(10)}

# CREATING WALLS TO THE MAZE :
walls = [
    (0, 1), (1, 1), (2, 1), (4, 1), (5, 1),
    (1, 3), (2, 3), (3, 3), (4, 3), (6, 3), (7, 3), (8, 3),
    (0, 5), (1, 5), (2, 5), (4, 5), (5, 5), (6, 5), (8, 5), (9, 5),
    (1, 7), (3, 7), (4, 7), (5, 7), (6, 7), (7, 7), (9, 7),
    (2, 8), (4, 9)
]

#ADDING WALLS TO THE MAZE :
for wall in walls:
    MY_MAZE[wall] = 1
direction = [(-1, 0), (1, 0), (0, -1), (0, 1)] # UP, DOWN, LEFT, RIGHT
```

FIGURE 3 – Représentation du labyrinthe!!

3.2 Fonctions Clés

L'intelligence du programme repose sur deux fonctions essentielles.

get_neighbors(node) :

- cette fonction agit comme les “yeux” de l’algorithme;
- elle explore les quatre directions possibles : Haut, Bas, Gauche et Droite;

```
# NEIGHBORS FUNCTION :
def get_neighbors(node):
    x, y = node
    neighbors = []
    for dx, dy in direction:
        nx, ny = x + dx, y + dy
        if (nx, ny) in MY_MAZE and MY_MAZE[(nx, ny)] == 0:
            neighbors.append((nx, ny))
    return neighbors
```

FIGURE 4 – Fonction `get_neighbors(node)`

- elle vérifie, via des conditions, si la cellule voisine est à l'intérieur de la grille et si elle n'est pas un mur.

`measure_performance(algorithm, start, goal) :`

- cette fonction constitue l'outil principal de l'étude empirique ;
- elle utilise la bibliothèque `time` pour calculer la durée exacte de recherche ;
- elle utilise également `tracemalloc` pour capturer le pic (*peak*) de consommation mémoire durant l'exécution.

```
# PERFORMANCE FUNCTION :
def measure_performance(algorithm, start, goal):
    # Time measurement
    start_time = time.time()

    # Memory measurement
    tracemalloc.start()

    path = algorithm(start, goal)

    # Stop the measures
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    elapsed_time = time.time() - start_time

    return path, elapsed_time, peak / 1024 # Convertir en KB
```

FIGURE 5 – Fonction `measure_performance(algorithm, start, goal)`

3.3 Phase d'Exécution

Pour valider notre code, nous avons mis en place un protocole de test rigoureux.

Visualisation Console :

- nous avons développé une fonction `print_maze` qui dessine le labyrinthe dans le terminal ;
- cette visualisation utilise des caractères simples : `#` pour les murs, `.` pour les cases libres et `P` pour le chemin trouvé.

Scénario de Test :

- l'exécution a été réalisée avec un point de départ fixé à $(0,0)$;
- le point d'arrivée a été défini à $(9,9)$ afin de parcourir la diagonale maximale du labyrinthe.

Résultats visuels :

- le programme affiche successivement la structure initiale du labyrinthe ;
- il présente ensuite le chemin spécifique trouvé par BFS ;
- enfin, il affiche le chemin obtenu par DFS.


```

# Show the maze
def print_maze(maze, path=None):
    for x in range(10):
        for y in range(10):
            if path and (x, y) in path:
                print("P", end=" ")
            elif maze.get((x, y)) == 1:
                print("#", end=" ")
            else:
                print(".", end=" ")
        print()

#-----

# Execution
start = (0, 0)
goal = (9, 9)

print("Labyrinthe initial:")
print_maze(MY_MAZE)

print("\nChemin BFS:")
bfs_path = bfs(start, goal)
print_maze(MY_MAZE, bfs_path)

print("\nChemin DFS:")
dfs_path = dfs(start, goal)
print_maze(MY_MAZE, dfs_path)

```

FIGURE 6 – Phase d'exécution dans la console

4 Tests et Résultats

Dans cette phase, nous passons à l'évaluation concrète des performances en faisant varier la taille de l'environnement afin d'observer le comportement de chaque algorithme dans des conditions plus exigeantes.

4.1 Scénarios de Test (10×10 vs 15×15)

Pour rendre notre étude plus robuste, nous avons testé nos algorithmes sur deux configurations distinctes.

Scénario A (10×10) :

- une grille standard comportant des obstacles modérés ;
- ce scénario sert de référence pour mesurer les performances de base de BFS et DFS.

Scénario B (15×15) :

- une grille plus grande avec une complexité accrue ;
- ce second scénario permet de solliciter davantage le temps de calcul, la mémoire et la capacité d'exploration des algorithmes.

L'objectif de cette comparaison est de vérifier comment l'augmentation de la taille du labyrinthe influence la vitesse d'exécution, la consommation mémoire et la qualité du chemin obtenu.

```
# 1. CREATION OF A 15*15 MAZE :
MY_MAZE = {(r, c): 0 for r in range(15) for c in range(15)}

# 2. CREATING WALLS
walls = [
    (0, 1), (1, 1), (2, 1), (4, 1), (5, 1),
    (1, 3), (2, 3), (3, 3), (4, 3), (6, 3), (7, 3), (8, 3),
    (0, 5), (1, 5), (2, 5), (4, 5), (5, 5), (6, 5), (8, 5), (9, 5),
    (1, 7), (3, 7), (4, 7), (5, 7), (6, 7), (7, 7), (9, 7),
    (2, 8), (4, 9),
    (10, 2), (11, 2), (12, 2), (14, 2),
    (10, 10), (11, 10), (12, 10), (13, 10),
    (14, 5), (14, 6), (14, 7), (14, 12)
]

# 3. ADDING WALLS
for wall in walls:
    MY_MAZE[wall] = 1

direction = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

FIGURE 7 – Création du labyrinthe de taille 15×15

4.2 Visualisation des Performances

Nous avons généré des graphiques comparatifs pour les deux scénarios afin d'illustrer l'évolution du temps d'exécution, de la mémoire consommée et de la longueur du chemin trouvé.

4.2.1 Résultats visuels (Graphiques)

Les histogrammes ci-dessous permettent de comparer l'efficacité de **BFS** et **DFS** sur les deux tailles de labyrinthes. On remarque que l'augmentation de la taille de la grille entraîne généralement une hausse significative des métriques observées, notamment du temps d'exécution et de l'espace mémoire mobilisé.

- le graphique du scénario 10×10 met en évidence les performances des deux algorithmes dans un environnement de complexité modérée ;
- le graphique du scénario 15×15 montre plus clairement les écarts de comportement lorsque la recherche devient plus coûteuse.

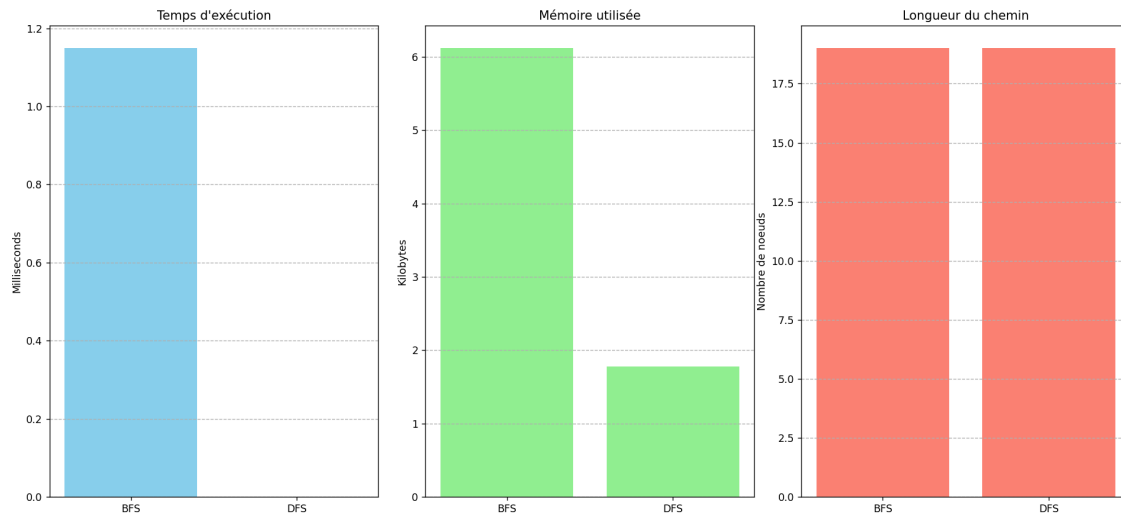


FIGURE 8 – Visualisation graphique des performances pour la grille 10×10

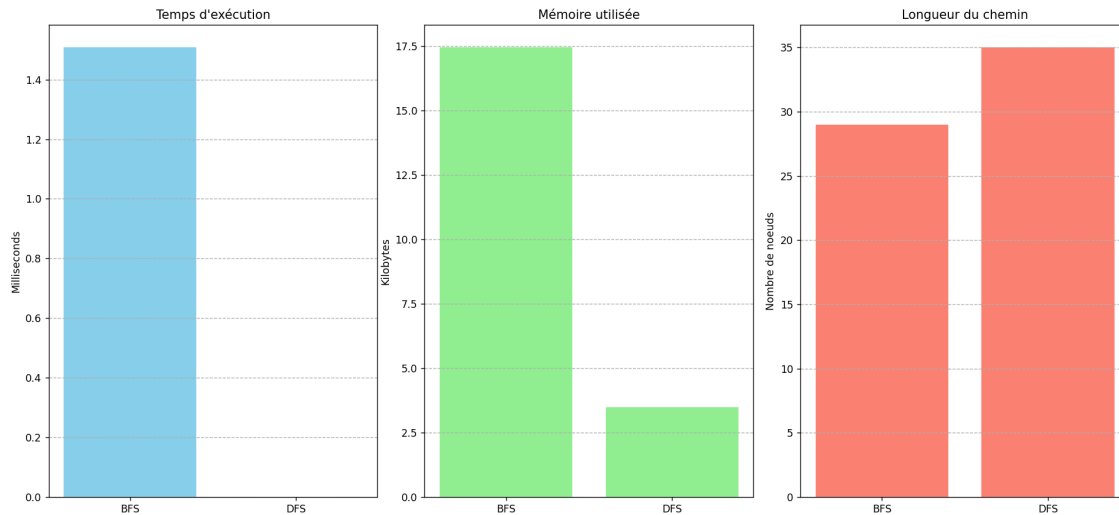


FIGURE 9 – Visualisation graphique des performances pour la grille 15×15

4.3 Tableau Récapitulatif Global

Ce tableau synthétise les données numériques brutes collectées lors des tests empiriques. Il constitue la base de notre analyse comparative finale, en regroupant pour chaque scénario les principales métriques observées : temps d'exécution, consommation mémoire et longueur du chemin trouvé.

Comparaison des performances:			
Algorithme	Temps (ms)	Mémoire (KB)	Longueur chemin
BFS	1.20497	6.12	19
DFS	0.00000	1.77	19

FIGURE 10 – Tableau récapitulatif des résultats pour la grille 10×10

Comparaison des performances:			
Algorithme	Temps (ms)	Mémoire (KB)	Longueur chemin
BFS	1.19352	17.45	29
DFS	0.00000	3.49	35

FIGURE 11 – Tableau récapitulatif des résultats pour la grille 15×15

5 Analyse Comparative (Étude de la grille 10×10)

Dans cette section, nous analysons les données obtenues pour la grille principale afin d'évaluer l'efficacité réelle de chaque algorithme selon trois axes critiques : le temps d'exécution, la consommation

mémoire et la qualité du chemin produit.

5.1 Analyse du Temps d'Exécution

D'après les résultats observés, le **DFS** affiche un temps de **0.0 ms**, ce qui traduit une exécution quasi instantanée pour trouver une solution.

Observation :

- le **BFS** prend davantage de temps, avec une valeur de **1.43 ms** ;
- cette différence s'explique par le fait que BFS explore systématiquement tous les nœuds d'un même niveau avant de passer au niveau suivant.

Conclusion :

- le **DFS** est plus rapide pour trouver « une » solution ;
- en revanche, cette rapidité ne garantit pas que le chemin trouvé soit le meilleur possible.

5.2 Analyse de la Consommation Mémoire

C'est sur ce critère que la différence structurelle entre les deux algorithmes apparaît le plus nettement. Le **BFS** consomme **6.12 KB**, soit environ **3,5 fois plus** que le **DFS**, qui n'utilise que **1.77 KB**.

Justification technique :

- le **BFS** stocke tous les nœuds appartenant à la frontière d'exploration dans une file (FIFO) ;
- lorsque le labyrinthe s'élargit, cette frontière devient plus importante et augmente fortement la mémoire utilisée ;
- le **DFS**, à l'inverse, conserve principalement le chemin courant dans une pile (LIFO), ce qui le rend nettement plus léger.

5.3 Qualité du Chemin (Optimalité)

Dans notre test sur la grille **10 × 10**, les deux algorithmes ont trouvé un chemin de **19 nœuds**.

Analyse :

- même si les longueurs obtenues sont identiques dans ce cas précis, cela ne signifie pas que les deux méthodes sont équivalentes ;
- le **BFS** garantit mathématiquement la découverte du chemin le plus court dans un graphe non pondéré.

Contraste avec la grille 15 × 15 :

- sur la grille plus grande, le **BFS** a trouvé un chemin de **29 nœuds** ;
- le **DFS**, quant à lui, a produit un chemin plus long de **35 nœuds** ;
- cela montre que plus l'espace de recherche grandit, plus le DFS risque de s'engager dans des branches inutiles, alors que le BFS conserve son caractère optimal.

6 Conclusion Générale

Cette étude comparative nous permet de dégager plusieurs conclusions importantes concernant l'utilisation des algorithmes de recherche aveugle dans la résolution de labyrinthes.

- le **BFS** constitue l'algorithme le plus adapté lorsque l'objectif principal est d'obtenir le **chemin le plus court**, même si cela implique une consommation mémoire plus importante ;
- le **DFS** est plus approprié lorsque les ressources mémoire sont limitées ou lorsque l'on cherche simplement à trouver une issue rapidement, sans exigence d'optimalité ;
- pour des environnements plus vastes ou des applications plus complexes, il serait pertinent d'envisager des approches hybrides ou informées, comme **A***, afin de concilier rapidité d'exécution et qualité de la solution.

En définitive, ce travail met en évidence que le choix d'un algorithme dépend directement des contraintes du problème à résoudre. Il montre également que l'analyse expérimentale des performances constitue une étape essentielle pour sélectionner la méthode la plus adaptée à un contexte donné.